Design Patterns

A Comprehensive Visual Guide

Real-Life Examples | C# | Python | Node.js

C# Python Node.js

Gang of Four (GoF) Design Patterns

Creational • Structural • Behavioral

15 Essential Patterns with Working Code Examples

15
Patterns

Table of Contents

Understanding Design Patterns

> **!** Design patterns are reusable solutions to common problems in software design. They provide a template for solving particular design problems in a specific context. Instead of reinventing the wheel, developers can use established patterns to create more efficient and maintainable code.

Evolution of Design Patterns

Design patterns have evolved over the years to address the increasing complexity of software development. The roots can be traced back to early software engineers who recognized the need for reusable solutions. The concept emerged in the context of object-oriented programming (OOP), where the focus was on creating reusable and maintainable code.

The Gang of Four (GoF)

The Gang of Four refers to the four authors of the seminal book *"Design Patterns: Elements of Reusable Object-Oriented Software"* (1994):

Author	Contribution
Erich Gamma	Lead author, Eclipse JDT architect
Richard Helm	Software engineering researcher
Ralph Johnson	University of Illinois professor
John Vlissides	IBM researcher, pattern advocate

GoF Contributions

The GoF provided a comprehensive catalog of 23 design patterns, categorizing them into three groups: **Creational** (object creation mechanisms), **Structural** (object composition), and **Behavioral** (object interaction and responsibility). Their work became the foundation of modern software design practices.

Types of Design Patterns

- **C** Creational Patterns: Deal with object creation and initialization, giving flexibility in deciding which objects need to be created. Patterns: Singleton, Factory Method, Abstract Factory, Builder, Prototype
- **S** Structural Patterns: Deal with class and object composition, focusing on decoupling interfaces and implementations. Patterns: Adapter, Bridge, Composite, Decorator, Facade
- **B** Behavioral Patterns: Deal with communication between objects, defining how they interact and communicate. Patterns: Observer, Strategy, Command, Iterator, State

Creational Patterns

Object creation mechanisms — flexibility in what gets created

1 Singleton

Ensures a class has only **one instance** and provides a global point of access to it. Like a country having only one president at a time.

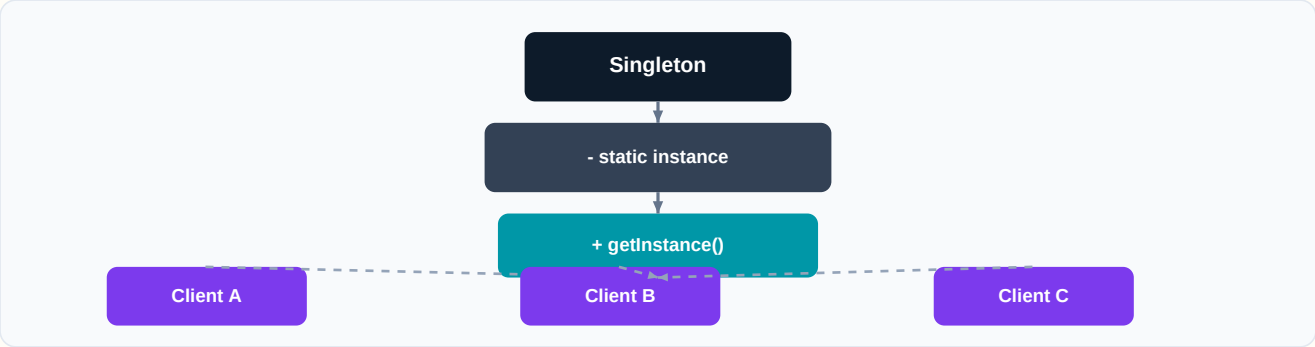
Real-Life: Government Office

A country has only one official government. No matter which citizen you ask, they all refer to the same government. The Singleton ensures only one instance exists globally.

Real-Life: Database Connection Pool

An application typically has one shared connection pool rather than creating a new pool every time — saving resources and ensuring consistency.

Visual Diagram



Code Examples

PYTHON

```python
class DatabaseConnection:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.connection = "Connected to DB"
        return cls._instance

# Usage
db1 = DatabaseConnection()
db2 = DatabaseConnection()
print(db1 is db2)  # True — same instance
```

C#

```csharp
public sealed class DatabaseConnection
{
    private static DatabaseConnection _instance;
    private static readonly object _lock = new object();
    public string Connection { get; private set; }

    private DatabaseConnection()
    {
        Connection = "Connected to DB";
    }

    public static DatabaseConnection Instance
    {
        get
        {
            lock (_lock)
            {
                if (_instance == null)
                    _instance = new DatabaseConnection();
                return _instance;
            }
        }
    }
}
// Usage: var db = DatabaseConnection.Instance;
```

NODE.JS

```javascript
class DatabaseConnection {
    constructor() {
        if (DatabaseConnection.instance) {
            return DatabaseConnection.instance;
        }
        this.connection = "Connected to DB";
        DatabaseConnection.instance = this;
    }
}

// Usage
const db1 = new DatabaseConnection();
const db2 = new DatabaseConnection();
console.log(db1 === db2); // true
```

2 Factory Method

Defines an interface for creating objects, but lets **subclasses decide** which class to instantiate. The factory method lets a class defer instantiation to subclasses.

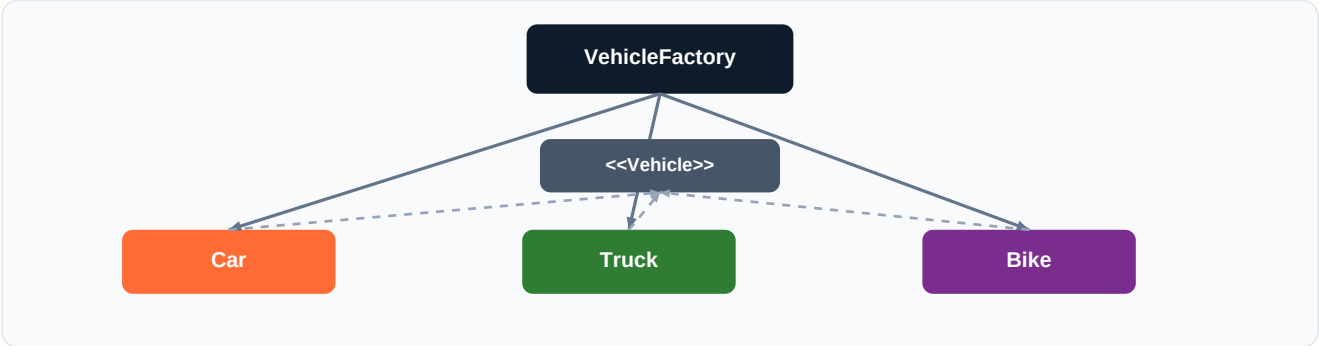
Real-Life: Vehicle Showroom

You walk into a dealership and say 'I want a vehicle.' The showroom (factory) decides whether to give you a Car, Truck, or Bike based on your needs — you never build it yourself.

Real-Life: Restaurant Kitchen

You order 'pasta' from the menu. The kitchen (factory) decides the exact recipe and preparation. You get a finished dish without knowing the creation details.

Visual Diagram



Code Examples

PYTHON

```python
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def deliver(self): pass

class Car(Vehicle):
    def deliver(self): return "Car delivered by road"

class Truck(Vehicle):
    def deliver(self): return "Truck delivered by freight"

class VehicleFactory:
    @staticmethod
    def create(vehicle_type):
        if vehicle_type == "car": return Car()
        if vehicle_type == "truck": return Truck()
        raise ValueError("Unknown type")

# Usage
vehicle = VehicleFactory.create("car")
print(vehicle.deliver())
```

C#

```
public interface IVehicle { string Deliver(); }

public class Car : IVehicle
{
    public string Deliver() => "Car delivered by road";
}
public class Truck : IVehicle
{
    public string Deliver() => "Truck delivered by freight";
}

public static class VehicleFactory
{
    public static IVehicle Create(string type) => type switch
    {
        "car"   => new Car(),
        "truck" => new Truck(),
        _ => throw new ArgumentException("Unknown")
    };
}
// var v = VehicleFactory.Create("car");
```

NODE.JS

```
class Car {
    deliver() { return "Car delivered by road"; }
}
class Truck {
    deliver() { return "Truck delivered by freight"; }
}

class VehicleFactory {
    static create(type) {
        switch (type) {
            case "car":   return new Car();
            case "truck": return new Truck();
            default: throw new Error("Unknown type");
        }
    }
}

const v = VehicleFactory.create("car");
console.log(v.deliver());
```

3 Abstract Factory

Provides an interface for creating **families of related objects** without specifying their concrete classes. Think of it as a factory of factories.

Real-Life: Furniture Store

IKEA has different collections — Modern, Victorian, Art Deco. Each collection (abstract factory) produces matching chairs, tables, and sofas that share a consistent style.

Real-Life: OS UI Toolkit

Windows and macOS each have their own set of buttons, scrollbars, and menus. An abstract factory creates the right family of UI widgets for each OS.

Visual Diagram

Each factory creates a family of related products

Code Examples

PYTHON

```python
from abc import ABC, abstractmethod

class Chair(ABC):
    @abstractmethod
    def sit_on(self): pass

class ModernChair(Chair):
    def sit_on(self): return "Sitting on modern chair"

class VictorianChair(Chair):
    def sit_on(self): return "Sitting on victorian chair"

class FurnitureFactory(ABC):
    @abstractmethod
    def create_chair(self): pass

class ModernFactory(FurnitureFactory):
    def create_chair(self): return ModernChair()

class VictorianFactory(FurnitureFactory):
    def create_chair(self): return VictorianChair()

# Usage
factory = ModernFactory()
chair = factory.create_chair()
print(chair.sit_on())
```

C#

```
public interface IChair { string SitOn(); }
public class ModernChair : IChair
{
    public string SitOn() => "Sitting on modern chair";
}
public class VictorianChair : IChair
{
    public string SitOn() => "Sitting on victorian chair";
}

public interface IFurnitureFactory
{
    IChair CreateChair();
}
public class ModernFactory : IFurnitureFactory
{
    public IChair CreateChair() => new ModernChair();
}
public class VictorianFactory : IFurnitureFactory
{
    public IChair CreateChair() => new VictorianChair();
}
// IFurnitureFactory f = new ModernFactory();
```

NODE.JS

```
class ModernChair {
    sitOn() { return "Sitting on modern chair"; }
}
class VictorianChair {
    sitOn() { return "Sitting on victorian chair"; }
}
class ModernFactory {
    createChair() { return new ModernChair(); }
}
class VictorianFactory {
    createChair() { return new VictorianChair(); }
}

const factory = new ModernFactory();
console.log(factory.createChair().sitOn());
```

4 Builder

Separates the **construction of a complex object** from its representation, allowing the same construction process to create different representations.

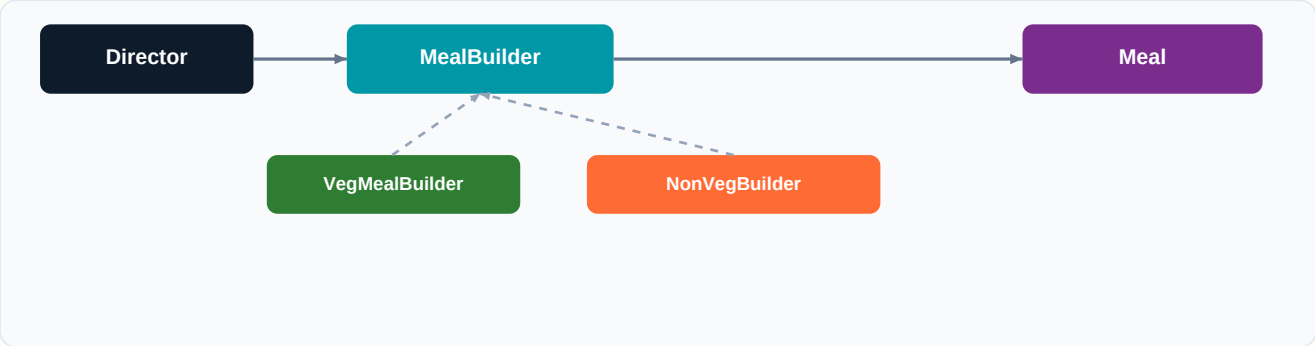
Real-Life: Subway Sandwich

At Subway, you build your sandwich step-by-step: choose bread, then protein, then veggies, then sauce. The Builder pattern lets you construct complex objects one step at a time.

Real-Life: House Construction

A construction director tells the builder to lay foundation, build walls, add roof, and install plumbing — each step is independent, and different builders produce different house styles.

Visual Diagram



Code Examples

PYTHON

```python
class Meal:
    def __init__(self):
        self.items = []
    def add_item(self, item):
        self.items.append(item)
    def show(self):
        return ", ".join(self.items)

class MealBuilder:
    def __init__(self):
        self.meal = Meal()
    def add_burger(self):
        self.meal.add_item("Veg Burger"); return self
    def add_drink(self):
        self.meal.add_item("Coke"); return self
    def add_fries(self):
        self.meal.add_item("Fries"); return self
    def build(self):
        return self.meal

meal = MealBuilder().add_burger().add_drink().add_fries().build()
print(meal.show())  # Veg Burger, Coke, Fries
```

C#

```
public class Meal
{
    private List<string> _items = new();
    public void AddItem(string item) => _items.Add(item);
    public string Show() => string.Join(", ", _items);
}

public class MealBuilder
{
    private Meal _meal = new();
    public MealBuilder AddBurger()
    { _meal.AddItem("Veg Burger"); return this; }
    public MealBuilder AddDrink()
    { _meal.AddItem("Coke"); return this; }
    public MealBuilder AddFries()
    { _meal.AddItem("Fries"); return this; }
    public Meal Build() => _meal;
}

// var meal = new MealBuilder()
//     .AddBurger().AddDrink().AddFries().Build();
```

NODE.JS

```
class Meal {
    constructor() { this.items = []; }
    addItem(item) { this.items.push(item); }
    show() { return this.items.join(", "); }
}

class MealBuilder {
    constructor() { this.meal = new Meal(); }
    addBurger() { this.meal.addItem("Veg Burger"); return this; }
    addDrink()  { this.meal.addItem("Coke"); return this; }
    addFries()  { this.meal.addItem("Fries"); return this; }
    build()     { return this.meal; }
}

const meal = new MealBuilder()
    .addBurger().addDrink().addFries().build();
console.log(meal.show());
```

5 Prototype

Creates new objects by **cloning an existing object** (the prototype), avoiding the cost of creating objects from scratch.

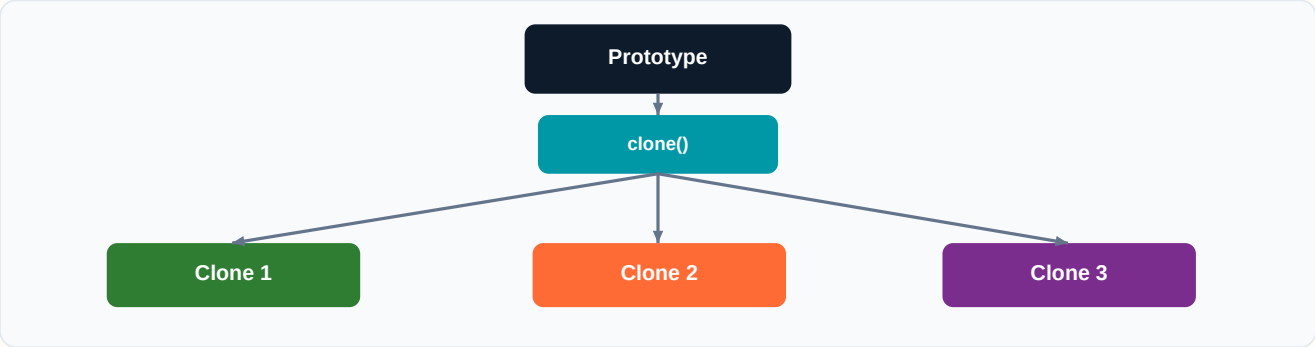
Real-Life: Cell Division

In biology, cells replicate by splitting — the new cell is a clone of the original. Similarly, the Prototype pattern clones existing objects to create new ones efficiently.

Real-Life: Document Templates

Instead of creating a new resume from scratch, you copy a template and modify it. The template is the prototype you clone.

Visual Diagram



Code Examples

PYTHON

```python
import copy

class Document:
    def __init__(self, title, content):
        self.title = title
        self.content = content

    def clone(self):
        return copy.deepcopy(self)

# Usage
template = Document("Resume", "Default content...")
my_resume = template.clone()
my_resume.title = "My Resume"
print(my_resume.title)   # My Resume
print(template.title)    # Resume (unchanged)
```

C#

```
public class Document : ICloneable
{
    public string Title { get; set; }
    public string Content { get; set; }

    public object Clone()
    {
        return this.MemberwiseClone();
    }
}

// Usage
var template = new Document
{
    Title = "Resume",
    Content = "Default content..."
};
var myResume = (Document)template.Clone();
myResume.Title = "My Resume";
// template.Title is still "Resume"
```

NODE.JS

```
class Document {
    constructor(title, content) {
        this.title = title;
        this.content = content;
    }
    clone() {
        return Object.assign(
            Object.create(Object.getPrototypeOf(this)),
            JSON.parse(JSON.stringify(this))
        );
    }
}

const template = new Document("Resume", "Default...");
const myResume = template.clone();
myResume.title = "My Resume";
console.log(template.title);  // Resume
console.log(myResume.title);  // My Resume
```

Structural Patterns

Object composition — how classes and objects are combined

6 Adapter

Converts the interface of a class into **another interface** that clients expect. Lets classes work together that couldn't otherwise because of incompatible interfaces.

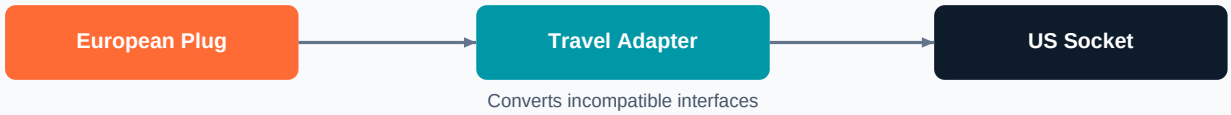
Real-Life: Travel Power Adapter

When you travel to a country with different plug sockets, you use a travel adapter. It doesn't change the electricity — it converts the plug shape so your device fits.

Real-Life: Language Translator

A translator at the UN converts one language into another so delegates can communicate — without either side needing to learn a new language.

Visual Diagram



Code Examples

PYTHON

```python
class EuropeanPlug:
    def provide_220v(self):
        return "220V power from EU plug"

class USSocket:
    def provide_110v(self):
        return "110V power from US socket"

class Adapter:
    def __init__(self, us_socket):
        self.us_socket = us_socket

    def provide_220v(self):
        power = self.us_socket.provide_110v()
        return f"Adapted: {power} -> 220V"

adapter = Adapter(USSocket())
print(adapter.provide_220v())
```

C#

```
public interface IEuropeanPlug
{
    string Provide220V();
}

public class USSocket
{
    public string Provide110V() => "110V from US socket";
}

public class SocketAdapter : IEuropeanPlug
{
    private readonly USSocket _usSocket;
    public SocketAdapter(USSocket s) => _usSocket = s;

    public string Provide220V()
    {
        var power = _usSocket.Provide110V();
        return $"Adapted: {power} -> 220V";
    }
}
// IEuropeanPlug plug = new SocketAdapter(new USSocket());
```

NODE.JS

```
class USSocket {
    provide110V() { return "110V from US socket"; }
}

class SocketAdapter {
    constructor(usSocket) { this.usSocket = usSocket; }

    provide220V() {
        const power = this.usSocket.provide110V();
        return `Adapted: ${power} -> 220V`;
    }
}

const adapter = new SocketAdapter(new USSocket());
console.log(adapter.provide220V());
```

7 Bridge

Decouples an **abstraction from its implementation** so the two can vary independently. Like separating a remote control from the TV it operates.

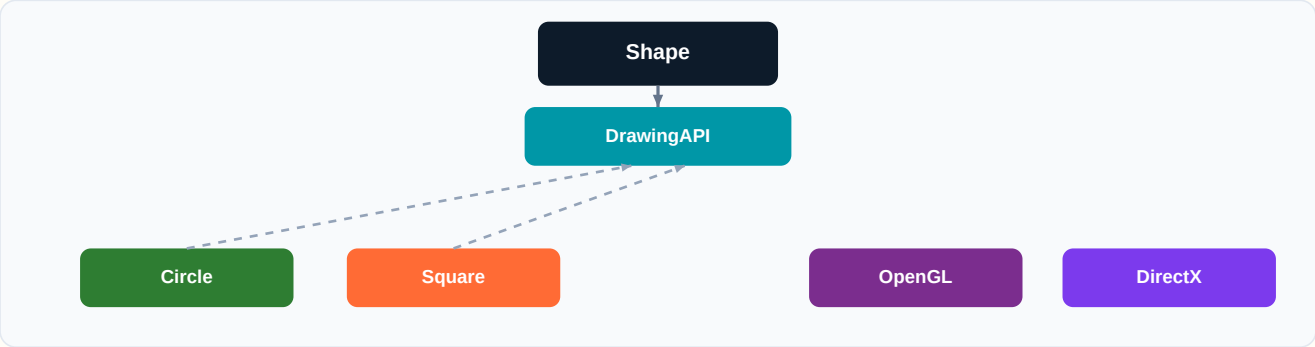
Real-Life: Remote Control + TV

A universal remote (abstraction) works with any TV brand (implementation). You can swap the TV without changing the remote, and upgrade the remote without touching the TV.

Real-Life: Shapes + Drawing APIs

A drawing app has shapes (Circle, Square) and rendering engines (OpenGL, DirectX). Bridge lets you combine any shape with any renderer independently.

Visual Diagram



Code Examples

PYTHON

```python
from abc import ABC, abstractmethod

class DrawingAPI(ABC):
    @abstractmethod
    def draw_circle(self, x, y, r): pass

class OpenGLAPI(DrawingAPI):
    def draw_circle(self, x, y, r):
        return f"OpenGL circle at ({x},{y}) r={r}"

class DirectXAPI(DrawingAPI):
    def draw_circle(self, x, y, r):
        return f"DirectX circle at ({x},{y}) r={r}"

class Shape:
    def __init__(self, api: DrawingAPI):
        self.api = api

class Circle(Shape):
    def __init__(self, x, y, r, api):
        super().__init__(api)
        self.x, self.y, self.r = x, y, r
    def draw(self):
        return self.api.draw_circle(self.x, self.y, self.r)

c = Circle(5, 10, 7, OpenGLAPI())
print(c.draw())
```

C#

```
public interface IDrawingAPI
{
    string DrawCircle(int x, int y, int r);
}
public class OpenGLAPI : IDrawingAPI
{
    public string DrawCircle(int x, int y, int r)
        => $"OpenGL circle at ({x},{y}) r={r}";
}

public abstract class Shape
{
    protected IDrawingAPI _api;
    protected Shape(IDrawingAPI api) => _api = api;
    public abstract string Draw();
}

public class Circle : Shape
{
    private int _x, _y, _r;
    public Circle(int x, int y, int r, IDrawingAPI api)
        : base(api) { _x=x; _y=y; _r=r; }
    public override string Draw()
        => _api.DrawCircle(_x, _y, _r);
}
```

NODE.JS

```
class OpenGLAPI {
    drawCircle(x, y, r) {
        return `OpenGL circle at (${x},${y}) r=${r}`;
    }
}
class DirectXAPI {
    drawCircle(x, y, r) {
        return `DirectX circle at (${x},${y}) r=${r}`;
    }
}

class Circle {
    constructor(x, y, r, api) {
        this.x = x; this.y = y;
        this.r = r; this.api = api;
    }
    draw() {
        return this.api.drawCircle(this.x, this.y, this.r);
    }
}

const c = new Circle(5, 10, 7, new OpenGLAPI());
console.log(c.draw());
```

8 Composite

Composes objects into **tree structures** to represent part-whole hierarchies. Lets clients treat individual objects and compositions uniformly.

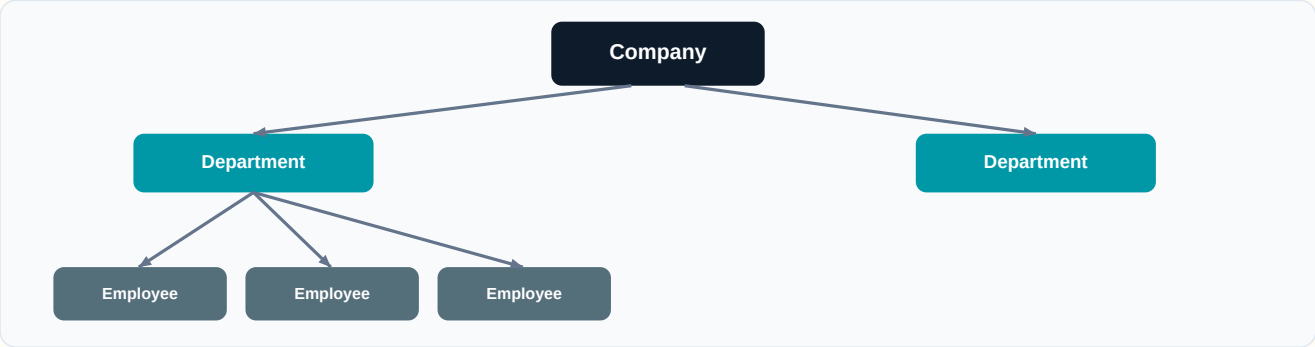
Real-Life: Company Org Chart

A company has departments, which have teams, which have employees. You can ask any node for its total salary — whether it's one person or the entire company.

Real-Life: File System

Folders contain files and other folders. Whether you delete a single file or a folder (and its contents), the operation interface is the same.

Visual Diagram



Code Examples

PYTHON

```python
from abc import ABC, abstractmethod

class Component(ABC):
    @abstractmethod
    def get_salary(self): pass

class Employee(Component):
    def __init__(self, name, salary):
        self.name = name; self.salary = salary
    def get_salary(self):
        return self.salary

class Department(Component):
    def __init__(self, name):
        self.name = name; self.members = []
    def add(self, component):
        self.members.append(component)
    def get_salary(self):
        return sum(m.get_salary() for m in self.members)

eng = Department("Engineering")
eng.add(Employee("Alice", 90000))
eng.add(Employee("Bob", 85000))
print(f"Dept salary: {eng.get_salary()}")  # 175000
```

C#

```
public interface IComponent { decimal GetSalary(); }

public class Employee : IComponent
{
    public string Name { get; }
    public decimal Salary { get; }
    public Employee(string n, decimal s)
    { Name = n; Salary = s; }
    public decimal GetSalary() => Salary;
}

public class Department : IComponent
{
    public string Name { get; }
    private List<IComponent> _members = new();
    public Department(string n) => Name = n;
    public void Add(IComponent c) => _members.Add(c);
    public decimal GetSalary()
        => _members.Sum(m => m.GetSalary());
}
```

NODE.JS

```
class Employee {
    constructor(name, salary) {
        this.name = name; this.salary = salary;
    }
    getSalary() { return this.salary; }
}

class Department {
    constructor(name) {
        this.name = name; this.members = [];
    }
    add(component) { this.members.push(component); }
    getSalary() {
        return this.members.reduce(
            (sum, m) => sum + m.getSalary(), 0
        );
    }
}

const eng = new Department("Engineering");
eng.add(new Employee("Alice", 90000));
eng.add(new Employee("Bob", 85000));
console.log(eng.getSalary()); // 175000
```

9 Decorator

Attaches additional responsibilities to an object **dynamically**. Provides a flexible alternative to subclassing for extending functionality.

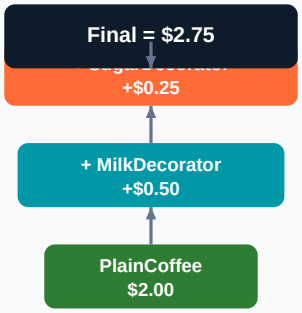
Real-Life: Coffee Shop Customization

Start with a plain coffee ($2). Add milk (+$0.50), add sugar (+$0.25), add whipped cream (+$0.75). Each add-on decorates the base coffee.

Real-Life: Gift Wrapping

A plain gift box can be decorated with ribbon, wrapping paper, and a bow. Each layer adds functionality without changing the core gift.

Visual Diagram



Code Examples

PYTHON

```python
from abc import ABC, abstractmethod

class Coffee(ABC):
    @abstractmethod
    def cost(self): pass
    @abstractmethod
    def description(self): pass

class SimpleCoffee(Coffee):
    def cost(self): return 2.00
    def description(self): return "Simple coffee"

class MilkDecorator(Coffee):
    def __init__(self, coffee):
        self._coffee = coffee
    def cost(self): return self._coffee.cost() + 0.50
    def description(self):
        return self._coffee.description() + " + milk"

class SugarDecorator(Coffee):
    def __init__(self, coffee):
        self._coffee = coffee
    def cost(self): return self._coffee.cost() + 0.25
    def description(self):
        return self._coffee.description() + " + sugar"

c = SugarDecorator(MilkDecorator(SimpleCoffee()))
print(f"{c.description()}: ${c.cost()}")
```

C#

```
public interface ICoffee
{
    decimal Cost();
    string Description();
}
public class SimpleCoffee : ICoffee
{
    public decimal Cost() => 2.00m;
    public string Description() => "Simple coffee";
}
public class MilkDecorator : ICoffee
{
    private ICoffee _coffee;
    public MilkDecorator(ICoffee c) => _coffee = c;
    public decimal Cost() => _coffee.Cost() + 0.50m;
    public string Description()
        => _coffee.Description() + " + milk";
}
public class SugarDecorator : ICoffee
{
    private ICoffee _coffee;
    public SugarDecorator(ICoffee c) => _coffee = c;
    public decimal Cost() => _coffee.Cost() + 0.25m;
    public string Description()
        => _coffee.Description() + " + sugar";
}
```

NODE.JS

```
class SimpleCoffee {
    cost() { return 2.00; }
    description() { return "Simple coffee"; }
}

class MilkDecorator {
    constructor(coffee) { this.coffee = coffee; }
    cost() { return this.coffee.cost() + 0.50; }
    description() {
        return this.coffee.description() + " + milk";
    }
}

class SugarDecorator {
    constructor(coffee) { this.coffee = coffee; }
    cost() { return this.coffee.cost() + 0.25; }
    description() {
        return this.coffee.description() + " + sugar";
    }
}

const c = new SugarDecorator(
    new MilkDecorator(new SimpleCoffee())
);
console.log(`${c.description()}: $${c.cost()}`);
```

10 Facade

Provides a **simplified interface** to a complex subsystem. Hides the complexity behind a simple API.

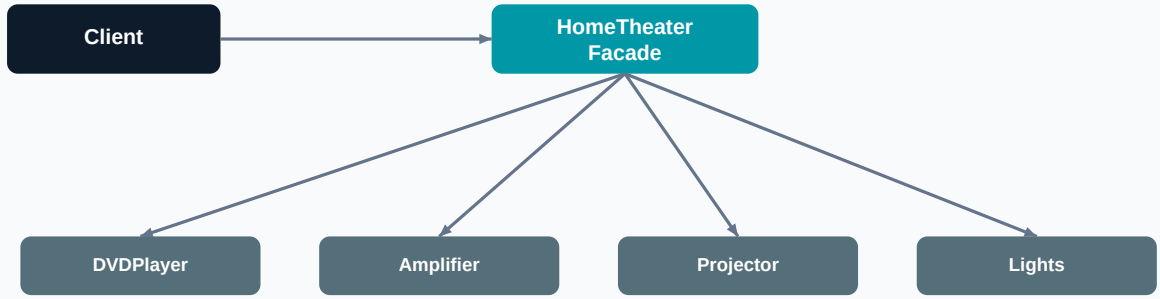
Real-Life: Home Theater System

Instead of turning on the projector, dimming lights, switching input, and adjusting volume separately — press one button on the remote. The Facade wraps the complex subsystem.

Real-Life: Hotel Concierge

You tell the concierge 'Plan my evening.' They coordinate restaurant reservation, taxi booking, and theater tickets — all behind a single request.

Visual Diagram



Code Examples

PYTHON

```python
class DVDPlayer:
    def on(self): return "DVD Player ON"
    def play(self, movie): return f"Playing {movie}"

class Amplifier:
    def on(self): return "Amplifier ON"
    def set_volume(self, v): return f"Volume set to {v}"

class Projector:
    def on(self): return "Projector ON"

class HomeTheaterFacade:
    def __init__(self):
        self.dvd = DVDPlayer()
        self.amp = Amplifier()
        self.proj = Projector()

    def watch_movie(self, movie):
        self.proj.on()
        self.amp.on()
        self.amp.set_volume(8)
        return self.dvd.play(movie)

theater = HomeTheaterFacade()
print(theater.watch_movie("Inception"))
```

C#

```
public class DVDPlayer
{
    public void On() => Console.WriteLine("DVD ON");
    public void Play(string m) => Console.WriteLine($"Playing {m}");
}
public class Amplifier
{
    public void On() => Console.WriteLine("Amp ON");
    public void SetVolume(int v) => Console.WriteLine($"Vol: {v}");
}
public class Projector
{
    public void On() => Console.WriteLine("Projector ON");
}

public class HomeTheaterFacade
{
    private DVDPlayer _dvd = new();
    private Amplifier _amp = new();
    private Projector _proj = new();

    public void WatchMovie(string movie)
    {
        _proj.On(); _amp.On();
        _amp.SetVolume(8); _dvd.Play(movie);
    }
}
```

NODE.JS

```
class DVDPlayer {
    on() { console.log("DVD ON"); }
    play(movie) { console.log(`Playing ${movie}`); }
}
class Amplifier {
    on() { console.log("Amp ON"); }
    setVolume(v) { console.log(`Volume: ${v}`); }
}
class Projector {
    on() { console.log("Projector ON"); }
}

class HomeTheaterFacade {
    constructor() {
        this.dvd = new DVDPlayer();
        this.amp = new Amplifier();
        this.proj = new Projector();
    }
    watchMovie(movie) {
        this.proj.on(); this.amp.on();
        this.amp.setVolume(8);
        this.dvd.play(movie);
    }
}

new HomeTheaterFacade().watchMovie("Inception");
```

Behavioral Patterns

Object interaction — how objects communicate and behave

11 Observer

Defines a **one-to-many dependency** between objects so that when one object changes state, all its dependents are notified and updated automatically.

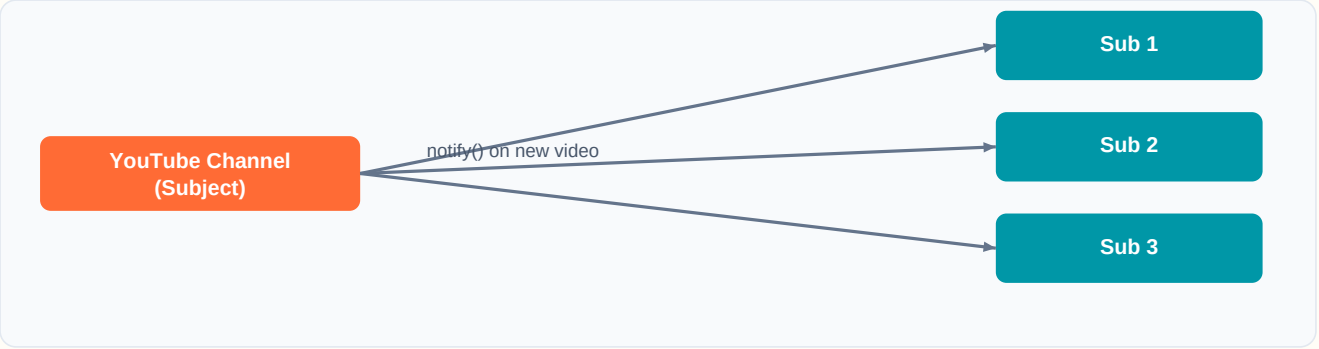
Real-Life: YouTube Subscriptions

When you subscribe to a channel, you get notified of every new video. The channel is the Subject, and subscribers are Observers. Unsubscribe = stop getting updates.

Real-Life: Newspaper Delivery

Subscribe to a newspaper and it arrives daily. The publisher doesn't care how many subscribers there are — it just sends papers to everyone on the list.

Visual Diagram



Code Examples

PYTHON

```python
class YouTubeChannel:
    def __init__(self, name):
        self.name = name
        self._subscribers = []
    def subscribe(self, sub):
        self._subscribers.append(sub)
    def notify(self, video):
        for sub in self._subscribers:
            sub.update(self.name, video)

class Subscriber:
    def __init__(self, name):
        self.name = name
    def update(self, channel, video):
        print(f"{self.name}: New video '{video}' on {channel}")

ch = YouTubeChannel("TechTalks")
ch.subscribe(Subscriber("Alice"))
ch.subscribe(Subscriber("Bob"))
ch.notify("Design Patterns Explained")
```

C#

```
public class YouTubeChannel
{
    public string Name { get; }
    private List<ISubscriber> _subs = new();
    public YouTubeChannel(string n) => Name = n;
    public void Subscribe(ISubscriber s) => _subs.Add(s);
    public void Notify(string video)
    {
        foreach (var s in _subs)
            s.Update(Name, video);
    }
}

public interface ISubscriber
{
    void Update(string channel, string video);
}

public class Subscriber : ISubscriber
{
    public string Name { get; }
    public Subscriber(string n) => Name = n;
    public void Update(string ch, string video)
        => Console.WriteLine($"{Name}: '{video}' on {ch}");
}
```

NODE.JS

```
class YouTubeChannel {
    constructor(name) {
        this.name = name;
        this.subscribers = [];
    }
    subscribe(sub) { this.subscribers.push(sub); }
    notify(video) {
        this.subscribers.forEach(sub =>
            sub.update(this.name, video)
        );
    }
}

class Subscriber {
    constructor(name) { this.name = name; }
    update(channel, video) {
        console.log(`${this.name}: '${video}' on ${channel}`);
    }
}

const ch = new YouTubeChannel("TechTalks");
ch.subscribe(new Subscriber("Alice"));
ch.subscribe(new Subscriber("Bob"));
ch.notify("Design Patterns Explained");
```

12 Strategy

Defines a family of algorithms, encapsulates each one, and makes them **interchangeable**. Lets the algorithm vary independently from clients that use it.

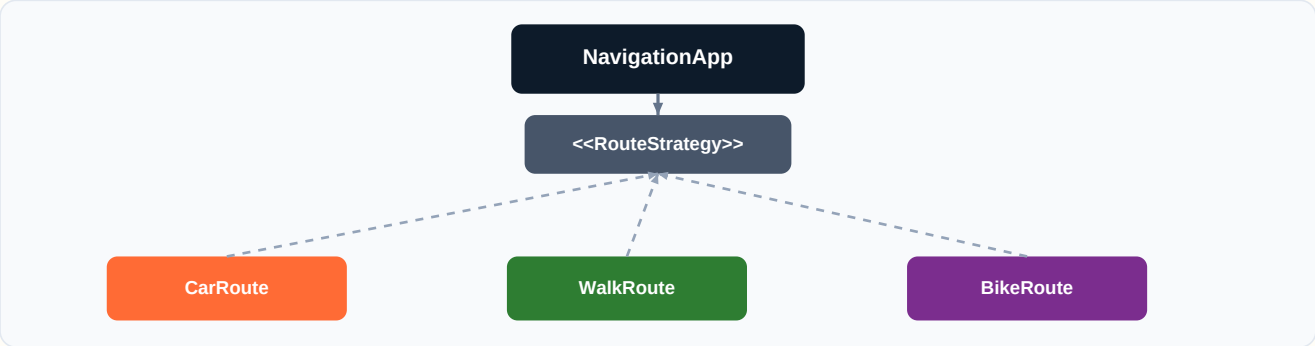
Real-Life: GPS Navigation

Google Maps offers multiple route strategies: fastest by car, walking, cycling, public transit. The destination is the same, but the strategy (algorithm) for getting there changes.

Real-Life: Payment Methods

An e-commerce site supports credit card, PayPal, and crypto. The checkout process is the same — only the payment strategy changes.

Visual Diagram



Code Examples

PYTHON

```python
from abc import ABC, abstractmethod

class RouteStrategy(ABC):
    @abstractmethod
    def calculate(self, origin, dest): pass

class CarRoute(RouteStrategy):
    def calculate(self, origin, dest):
        return f"Driving from {origin} to {dest} via highway"

class WalkRoute(RouteStrategy):
    def calculate(self, origin, dest):
        return f"Walking from {origin} to {dest} via park"

class Navigator:
    def __init__(self, strategy: RouteStrategy):
        self._strategy = strategy

    def navigate(self, origin, dest):
        return self._strategy.calculate(origin, dest)

nav = Navigator(CarRoute())
print(nav.navigate("Home", "Office"))
nav = Navigator(WalkRoute())
print(nav.navigate("Home", "Park"))
```

C#

```
public interface IRouteStrategy
{
    string Calculate(string origin, string dest);
}
public class CarRoute : IRouteStrategy
{
    public string Calculate(string o, string d)
        => $"Driving {o} to {d} via highway";
}
public class WalkRoute : IRouteStrategy
{
    public string Calculate(string o, string d)
        => $"Walking {o} to {d} via park";
}

public class Navigator
{
    private IRouteStrategy _strategy;
    public Navigator(IRouteStrategy s) => _strategy = s;
    public string Navigate(string o, string d)
        => _strategy.Calculate(o, d);
}
// var nav = new Navigator(new CarRoute());
```

NODE.JS

```
class CarRoute {
    calculate(origin, dest) {
        return `Driving ${origin} to ${dest} via highway`;
    }
}
class WalkRoute {
    calculate(origin, dest) {
        return `Walking ${origin} to ${dest} via park`;
    }
}

class Navigator {
    constructor(strategy) { this.strategy = strategy; }
    navigate(origin, dest) {
        return this.strategy.calculate(origin, dest);
    }
}

let nav = new Navigator(new CarRoute());
console.log(nav.navigate("Home", "Office"));
nav = new Navigator(new WalkRoute());
console.log(nav.navigate("Home", "Park"));
```

13 Command

Encapsulates a request as an **object**, letting you parameterize clients with different requests, queue or log requests, and support undoable operations.

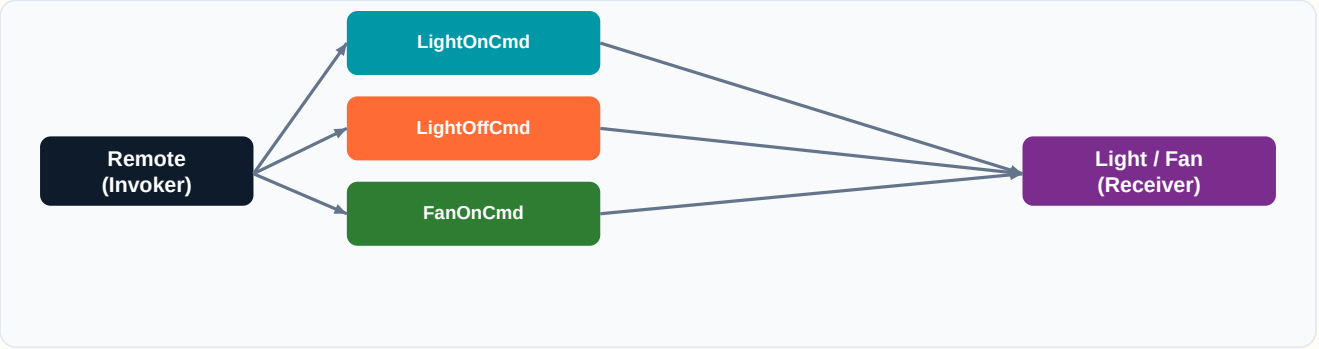
Real-Life: Restaurant Order Slip

A waiter writes your order on a slip (command object). The chef executes it later. Orders can be queued, modified, or cancelled — all without the waiter knowing how to cook.

Real-Life: TV Remote Buttons

Each button on a remote is a command. Press 'Volume Up' and it encapsulates the request to the TV. The remote doesn't know how volume works internally.

Visual Diagram



Code Examples

PYTHON

```python
from abc import ABC, abstractmethod

class Command(ABC):
    @abstractmethod
    def execute(self): pass

class Light:
    def on(self): return "Light is ON"
    def off(self): return "Light is OFF"

class LightOnCommand(Command):
    def __init__(self, light):
        self.light = light
    def execute(self):
        return self.light.on()

class RemoteControl:
    def __init__(self):
        self._commands = []
    def add_command(self, cmd):
        self._commands.append(cmd)
    def execute_all(self):
        return [cmd.execute() for cmd in self._commands]

remote = RemoteControl()
light = Light()
remote.add_command(LightOnCommand(light))
print(remote.execute_all())
```

C#

```
public interface ICommand { string Execute(); }

public class Light
{
    public string On() => "Light is ON";
    public string Off() => "Light is OFF";
}

public class LightOnCommand : ICommand
{
    private Light _light;
    public LightOnCommand(Light l) => _light = l;
    public string Execute() => _light.On();
}

public class RemoteControl
{
    private List<ICommand> _commands = new();
    public void AddCommand(ICommand c) => _commands.Add(c);
    public void ExecuteAll()
    {
        foreach (var c in _commands)
            Console.WriteLine(c.Execute());
    }
}
```

NODE.JS

```
class Light {
    on() { return "Light is ON"; }
    off() { return "Light is OFF"; }
}

class LightOnCommand {
    constructor(light) { this.light = light; }
    execute() { return this.light.on(); }
}

class RemoteControl {
    constructor() { this.commands = []; }
    addCommand(cmd) { this.commands.push(cmd); }
    executeAll() {
        return this.commands.map(c => c.execute());
    }
}

const remote = new RemoteControl();
remote.addCommand(new LightOnCommand(new Light()));
console.log(remote.executeAll());
```

14 Iterator

Provides a way to access elements of a collection **sequentially** without exposing its underlying representation.

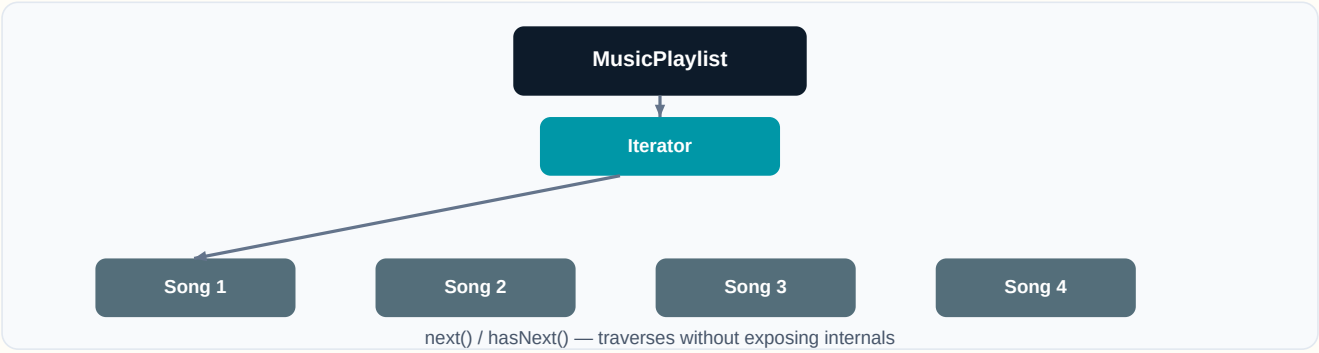
Real-Life: Music Playlist

When you hit 'Next' on Spotify, you iterate through a playlist. You don't care if songs are stored in an array, linked list, or database — you just get the next song.

Real-Life: TV Channel Surfing

Pressing the channel up/down button iterates through channels without you knowing how channels are internally stored or sorted.

Visual Diagram



next() / hasNext() — traverses without exposing internals

Code Examples

PYTHON

```python
class Playlist:
    def __init__(self):
        self._songs = []
    def add_song(self, song):
        self._songs.append(song)
    def __iter__(self):
        self._index = 0
        return self
    def __next__(self):
        if self._index >= len(self._songs):
            raise StopIteration
        song = self._songs[self._index]
        self._index += 1
        return song

playlist = Playlist()
playlist.add_song("Bohemian Rhapsody")
playlist.add_song("Stairway to Heaven")
playlist.add_song("Hotel California")

for song in playlist:
    print(f"Now playing: {song}")
```

C#

```csharp
public class Playlist : IEnumerable<string>
{
    private List<string> _songs = new();
    public void AddSong(string s) => _songs.Add(s);

    public IEnumerator<string> GetEnumerator()
    {
        foreach (var song in _songs)
            yield return song;
    }

    IEnumerator IEnumerable.GetEnumerator()
        => GetEnumerator();
}

// var pl = new Playlist();
// pl.AddSong("Bohemian Rhapsody");
// foreach (var song in pl)
//     Console.WriteLine($"Playing: {song}");
```

NODE.JS

```javascript
class Playlist {
    constructor() { this.songs = []; }
    addSong(song) { this.songs.push(song); }

    [Symbol.iterator]() {
        let index = 0;
        const songs = this.songs;
        return {
            next() {
                if (index < songs.length) {
                    return { value: songs[index++], done: false };
                }
                return { done: true };
            }
        };
    }
}

const pl = new Playlist();
pl.addSong("Bohemian Rhapsody");
pl.addSong("Stairway to Heaven");
for (const song of pl) {
    console.log(`Playing: ${song}`);
}
```

15 State

Allows an object to **alter its behavior** when its internal state changes. The object appears to change its class.

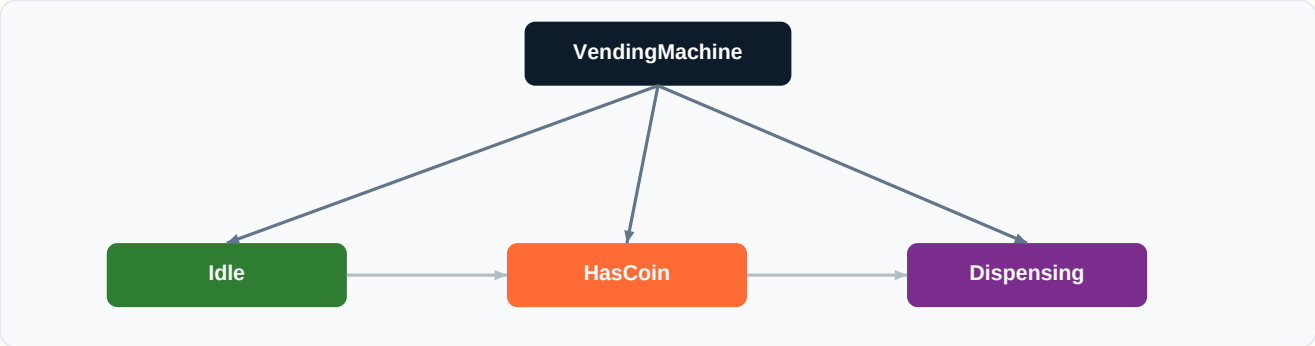
Real-Life: Vending Machine

A vending machine behaves differently depending on its state: waiting for coin, has coin inserted, dispensing product, or out of stock. Each state changes what buttons do.

Real-Life: Traffic Light

A traffic light cycles through Red, Yellow, and Green. In each state, it behaves differently (stop, caution, go) and transitions to the next state.

Visual Diagram



Code Examples

PYTHON

```python
from abc import ABC, abstractmethod

class State(ABC):
    @abstractmethod
    def insert_coin(self, machine): pass
    @abstractmethod
    def dispense(self, machine): pass

class IdleState(State):
    def insert_coin(self, machine):
        print("Coin inserted")
        machine.state = HasCoinState()
    def dispense(self, machine):
        print("Insert coin first!")

class HasCoinState(State):
    def insert_coin(self, machine):
        print("Coin already inserted")
    def dispense(self, machine):
        print("Dispensing item...")
        machine.state = IdleState()

class VendingMachine:
    def __init__(self):
        self.state = IdleState()
    def insert_coin(self): self.state.insert_coin(self)
    def dispense(self): self.state.dispense(self)

vm = VendingMachine()
vm.dispense()     # Insert coin first!
vm.insert_coin()   # Coin inserted
vm.dispense()     # Dispensing item...
```

C#

```csharp
public interface IState
{
    void InsertCoin(VendingMachine m);
    void Dispense(VendingMachine m);
}
public class IdleState : IState
{
    public void InsertCoin(VendingMachine m)
    {
        Console.WriteLine("Coin inserted");
        m.State = new HasCoinState();
    }
    public void Dispense(VendingMachine m)
        => Console.WriteLine("Insert coin first!");
}
public class HasCoinState : IState
{
    public void InsertCoin(VendingMachine m)
        => Console.WriteLine("Coin already inserted");
    public void Dispense(VendingMachine m)
    {
        Console.WriteLine("Dispensing...");
        m.State = new IdleState();
    }
}
public class VendingMachine
{
    public IState State { get; set; } = new IdleState();
    public void InsertCoin() => State.InsertCoin(this);
    public void Dispense() => State.Dispense(this);
}
```

NODE.JS

```
class IdleState {
    insertCoin(machine) {
        console.log("Coin inserted");
        machine.state = new HasCoinState();
    }
    dispense(machine) {
        console.log("Insert coin first!");
    }
}
class HasCoinState {
    insertCoin(machine) {
        console.log("Coin already inserted");
    }
    dispense(machine) {
        console.log("Dispensing...");
        machine.state = new IdleState();
    }
}

class VendingMachine {
    constructor() { this.state = new IdleState(); }
    insertCoin() { this.state.insertCoin(this); }
    dispense()   { this.state.dispense(this); }
}

const vm = new VendingMachine();
vm.dispense();     // Insert coin first!
vm.insertCoin();   // Coin inserted
vm.dispense();     // Dispensing...
```

Quick Reference — All 15 Patterns

#	Pattern	Category	Key Idea	Real-Life Analogy
1	Singleton	Creational	Ensures a class has only one instance and p...	Government Office
2	Factory Method	Creational	Defines an interface for creating objects, but let...	Vehicle Showroom
3	Abstract Factory	Creational	Provides an interface for creating families of ...	Furniture Store
4	Builder	Creational	Separates the construction of a complex object<...	Subway Sandwich
5	Prototype	Creational	Creates new objects by cloning an existing obje...	Cell Division
6	Adapter	Structural	Converts the interface of a class into another ...	Travel Adapter
7	Bridge	Structural	Decouples an abstraction from its implementatio...	Remote + TV
8	Composite	Structural	Composes objects into tree structures to re...	Org Chart
9	Decorator	Structural	Attaches additional responsibilities to an object ...	Coffee Customization
10	Facade	Structural	Provides a simplified interface to a comple...	Home Theater
11	Observer	Behavioral	Defines a one-to-many dependency between ob...	YouTube Subs
12	Strategy	Behavioral	Defines a family of algorithms, encapsulates each ...	GPS Navigation
13	Command	Behavioral	Encapsulates a request as an object, lettin...	Order Slip
14	Iterator	Behavioral	Provides a way to access elements of a collection ...	Music Playlist
15	State	Behavioral	Allows an object to alter its behavior when...	Vending Machine

Conclusion

The evolution of design patterns, spearheaded by the Gang of Four, has had a profound impact on software engineering. As technology continues to evolve, so too will the design patterns that help developers create efficient, maintainable, and scalable software solutions.

Understanding the history and development of design patterns is essential for any software developer looking to improve their design skills and knowledge. These 15 patterns — covering creational, structural, and behavioral categories — form the foundation that every developer should master.

* Remember: Design patterns are tools, not rules. Use them when they simplify your design. Over-engineering with patterns can be worse than having no patterns at all. Let the problem guide your choice of pattern.